FORME Project Architecture and Feature Guide

Technical implementation and application capabilities

Team reference | College web technology project | 5 November 2026

FORME is a clothing shopping application built with MongoDB, Express, React and Node.js. It combines a size-aware storefront and cash-on-delivery checkout with product reviews, outfit voting, measurement-based size guidance and suggestions that help shoppers complete an outfit.

This guide gives teammates a shared explanation of how the application works, where its main code lives and what to demonstrate. Part One explains the system architecture and technical implementation. Part Two explains the features and the combination of capabilities that distinguishes FORME within a college shopping-project demonstration.

Reading map

Part	Coverage	Pages
One	Architecture, frontend, backend, database and authentication	2 to 13
Two	Feature inventory, face-offs, reviews, sizing and cart suggestions	14 to 18
Team handover	Technical strengths, operation, validation and presentation guide	19 to 21

Project at a glance

- Two application roles: customer and administrator. Visitors can browse products, reviews and the community feed.
- 44 bundled sample products across Tops, Bottoms, Layers and Sets, with XS to XL stock variants and Indian Rupee pricing.
- One repository and dependency installation for the browser application, API, shared rules and maintenance scripts.
- Local password login plus optional Google sign-in through Neon Auth; shopping and community records remain in MongoDB.
- Current payment scope is cash on delivery. Size and outfit suggestions use explicit rules rather than trained AI models.

Code reference package.json; src/App.tsx; backend/models.ts; backend/catalog.ts; backend/campus-catalog.ts

Part One System architecture and implementation

1 System architecture

FORME follows a three-layer application structure: a React interface, an Express API and MongoDB persistence. The backend is a modular monolith: commerce, community, security and media are separate code modules inside one application, rather than independently deployed microservices.

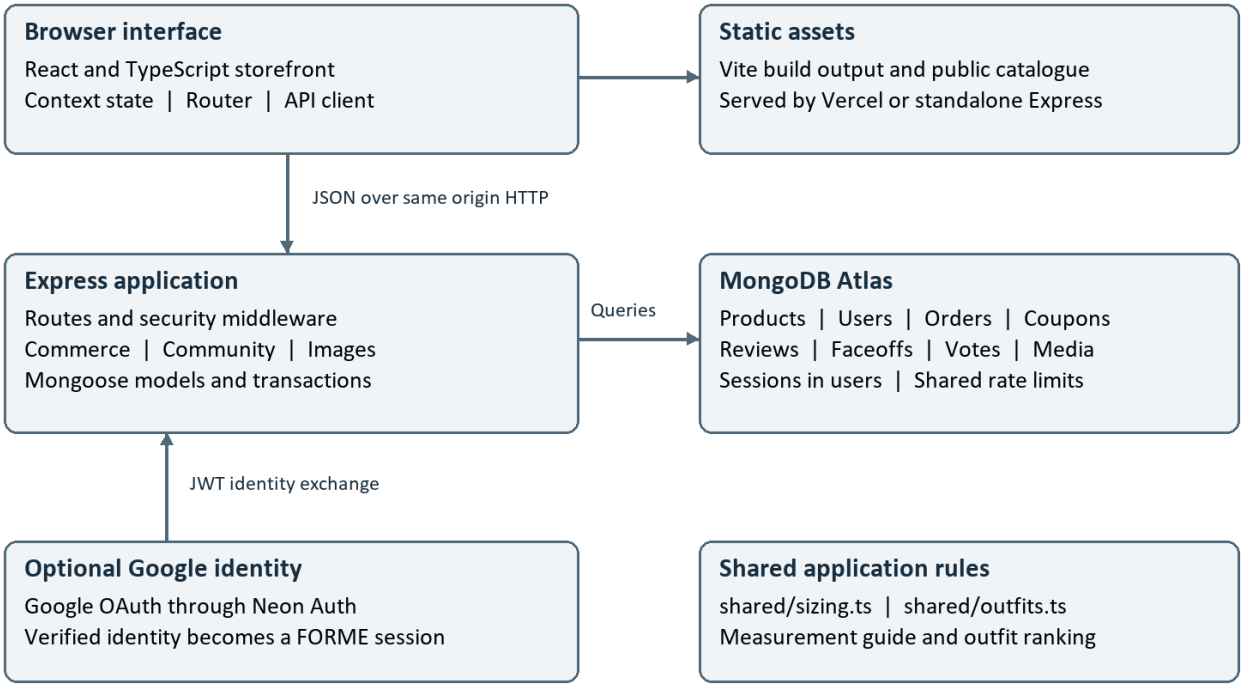


Figure 1 Main components and their relationships. Neon Auth supplies optional Google identity; MongoDB remains the application database. Shared rules are source modules, not a separate network service.

Responsibilities across the layers

- Browser: renders pages, collects input, keeps the device bag and wishlist, and sends JSON requests. It displays server results and helpful loading or error states.
- API: authenticates callers, checks ownership and roles, validates fields, calculates totals, reserves inventory and manages community content.
- Database: stores durable records and enforces unique indexes. Transactions coordinate order, stock, coupon and related community changes.
- Assets: product catalogue images ship with the site. Uploaded community photos are compressed and stored as binary data in MongoDB.

Code reference src/App.tsx; backend/app.ts; backend/database.ts; backend/media.ts; shared/sizing.ts; shared/outfits.ts

2 Runtime deployment and request flow

Local development and standalone production

server.ts creates one HTTP server around the Express app. In development it mounts Vite as middleware, so the interface and /api routes share an origin. Vite handles frontend updates and tsx watch restarts backend changes. The server starts from PORT or 3000; the current call allows one port attempt, so the team should stop an existing instance before starting another.

In standalone production, Express serves the Vite output in dist and returns index.html for application routes. Database connection failures keep the interface available while database operations return a temporary-unavailable response. The standalone process retries database readiness every 15 seconds.

Vercel deployment structure

vercel.json builds the Vite frontend into dist. The catch-all API function in api/[...path].ts passes requests into the Express application. The current backend/vercel.ts adapter removes Vercel catch-all query metadata before Express parses the actual URL. Non-API paths are rewritten to index.html so direct navigation to a product or account page reaches React Router.

Each warm API instance reuses a cached MongoDB connection promise with a maximum pool size of five. Vercel cold requests do not run catalogue migrations; database setup is a separate command. MongoDB counters and TTL indexes support rate limiting and draft cleanup without a scheduled application worker.

Example flow from product to order

1. React loads GET /api/products and renders the catalogue. A product detail request retrieves an active product by its productId.
2. The shopper selects a size. React checks available stock before adding productId, size and quantity to the local bag.
3. Checkout requests POST /api/checkout/quote using the authenticated cookie and CSRF header. The server reloads prices, validates stock and evaluates the coupon.
4. POST /api/orders repeats authoritative validation inside a transaction, reduces stock, redeems the coupon and creates an order snapshot.
5. React refreshes product stock and opens the order confirmation. An administrator later advances fulfilment status.

Code reference server.ts; api/[...path].ts; backend/vercel.ts; vercel.json; backend/database.ts; backend/commerce.ts

3 Frontend structure and main files

The frontend is a React single-page application written in TypeScript. src/main.tsx mounts App into the root element. App establishes routing, shared store access, the page shell and an error boundary. Components are grouped by shopping and account responsibilities under src/store.

File	Purpose
src/App.tsx	Route definitions, error boundary, shared header, footer, bag and login modal.
src/store/StoreContext.tsx	Products, session, bag, wishlist, login modal and notifications shared across pages.
src/store/Catalogue.tsx	Home page, product cards, filtering, sorting and product detail screen.
src/store/Layout.tsx	Header, responsive navigation, footer and shopping bag drawer.
src/store/Checkout.tsx	Shipping form, coupon quote, COD placement and confirmation.
src/store/Wardrobe.tsx	Size guide, complementary cart suggestions and review editor.
src/store/Community.tsx	Public feed, publishing form, voting and own-post view.
src/store/Account.tsx and Sessions.tsx	Customer dashboard, orders, profile, security and session controls.
src/store/Admin.tsx and Moderation.tsx	Store management, inventory, fulfilment, coupons and moderation.
src/store/Auth.tsx and GoogleCallback.tsx	Password forms, Google entry and identity exchange callback.
src/store/api.ts and types.ts	Common HTTP helper, API errors and shared interface types.
src/store/Photos.tsx and ui.tsx	Image selection and reusable modal, field, loading and image components.

Routes and navigation

The main paths are /, /shop, /product/:id, /community, /checkout, /order/:id, /account, /admin, /admin/community and /auth/callback. Unknown paths render the missing-page state. Customer and administrator pages check the loaded session before presenting their content; backend checks protect the actual data operations.

Code reference src/main.tsx; src/App.tsx; src/store/*

4 Frontend state management

FORME uses React Context and hooks for shared state. StoreProvider owns the active user, product list, bag, wishlist and interface flags. useStore exposes the same store to the header, catalogue, checkout and dashboards. This avoids passing those values through every intermediate component. Redux and Zustand are not used.

Where state lives

State	Where it lives	Examples
Shared application state	React Context with useState	User, products, bag, wishlist, notifications.
Page state	Local component hooks	Search text, selected size, form fields, current tab, loading state.
Device persistence	localStorage	forme_bag and forme_wishlist. These are not synchronized between devices.
Navigation state	Router URL and history state	Category and collection filters, saved-products view, buy-now data.
Authentication state	HttpOnly cookie plus server records	JWT cookie is sent by the browser; React receives a public user and CSRF token.

Code example of persistent bag state

```
const [bag, setBag] = useState<BagItem[]>(() =>
  stored('forme_bag').filter(i =>
    typeof i?.productId === 'string' &&
    ['XS', 'S', 'M', 'L', 'XL'].includes(i.size) &&
    Number.isInteger(i.quantity) &&
    i.quantity > 0 && i.quantity <= 10
  ).slice(0, 40)
);
useEffect(() => {
  localStorage.setItem('forme_bag', JSON.stringify(bag));
}, [bag]);
```

This excerpt initializes the bag once from browser storage, rejects malformed entries and saves later changes. A bag entry identifies a product and size rather than trusting a stored price. addToBag merges equal product-size entries, uses a functional state update and checks stock and the ten-unit limit. Checkout still validates everything on the server because localStorage can be edited.

On startup the provider loads products and GET /api/session. It also removes old token-related localStorage keys. When the API reports an expired protected session, the forme:expired event clears the public user and opens the login modal.

Code reference src/store/StoreContext.tsx; src/store/types.ts; src/store/api.ts

5 Frontend requests styling and interaction

One HTTP helper for the application

```
response = await fetch(url, {
  method,
  credentials: 'same-origin',
  headers: {
    'Content-Type': 'application/json',
    ...(csrfToken ? { 'X-CSRF-Token': csrfToken } : {})
  },
  ...(body !== undefined ? { body: JSON.stringify(body) } : {}),
  signal: AbortSignal.timeout(25000)
});
```

The shared api helper sends the HttpOnly cookie through the browser and adds the CSRF token retained in module memory. It parses JSON, preserves HTTP status in ApiError, updates the CSRF token returned by successful session operations and reports connection or timeout problems. Pages can therefore concentrate on their own form and display logic.

Catalogue and checkout interaction

The public product API returns active products, capped at 300. The shop uses that list for client-side category, collection, saved-item and text filtering, with featured or price sorting. Category, collection and wishlist-view choices are reflected in URL search parameters. There is no dedicated numeric price-range filter in the current Shop component.

Product details maintain selected size, quantity and modal state. Checkout supports the bag or a router-provided buy-now item. Effects request a fresh quote when items or coupons change; a cleanup flag prevents an obsolete response from overwriting current quote state. A request ID stays stable for retrying the same submission and changes when the order inputs change.

Styling and reusable interface behavior

- Vite loads React and Tailwind plugins. src/index.css imports Tailwind and the base, commerce, responsive, community and polish stylesheets.
- The visual design combines custom CSS, Motion interactions and Lucide icons. Reduced-motion handling appears in animations and reveal utilities.
- Reusable UI includes loading and empty states, notices, image fallbacks, labelled fields and dialogs. The page shell includes a skip-to-content link.
- Photos.tsx resizes chosen images in the browser before sending them. This reduces transfer size; authoritative image checks still run in Sharp on the server.

Code reference src/store/api.ts; src/store/Catalogue.tsx; src/store/Checkout.tsx; src/store/ui.tsx; src/index.css; src/utils/useScrollReveals.ts

6 Backend structure and middleware

The backend uses Express on Node.js with TypeScript modules executed through tsx. Routes orchestrate work; domain helpers validate business values; Mongoose models handle persistence. The code is organized into functional modules rather than a separate controllers and services directory structure.

Module	Responsibility
backend/app.ts	Creates Express, installs middleware and defines catalogue, account, order and admin APIs.
backend/commerce.ts	Calculates quotes, places orders and enforces fulfilment transitions.
backend/community.ts	Sizing endpoint, reviews, face-offs, votes, media routes and session endpoints.
backend/security.ts	JWT cookies, session checks, role checks, CSRF, origins, headers and rate limits.
backend/validation.ts and domain.ts	Request rules, injection-key rejection and validated business inputs.
backend/models.ts and database.ts	Schemas, indexes, connection cache and database initialization.
backend/media.ts and migrate-media.ts	Photo compression, ownership, attachment lifecycle and legacy migration.
backend/neon-auth.ts	Cryptographic Neon token validation and explicit account linking.
backend/privacy.ts and user-write.ts	Account erasure and transactional writes coordinated through the user record.
backend/catalog.ts and catalog-expansion.ts	Initial sample catalogue and repeatable campus catalogue installation.

How a request is handled

The app disables the Express identifying header, configures proxy trust for Vercel, adds security headers and checks mutation origins. API middleware ensures database readiness and applies a shared 300-requests-per-minute limiter. Uploads have their own JSON parser with a 4 MB limit; ordinary API JSON is limited to 100 KB. The recursive operator guard then rejects unsafe request keys.

Protected routes add authenticate, and mutations add csrf. Administrator routes add adminOnly. Async errors pass to a central handler that converts known business errors into useful HTTP responses and returns a generic message for unexpected failures. Duplicate keys become 409 responses; malformed JSON, invalid fields and oversized payloads receive appropriate errors.

Code reference backend/app.ts; backend/community.ts; backend/security.ts; backend/domain.ts; backend/validation.ts

7 Commerce logic and safe order placement

The server owns prices and totals

quoteCart accepts one to forty entries, merges repeated product-size pairs and limits each combined quantity to ten. It loads each active product, verifies the selected variant stock and builds item details from database values. Client-submitted prices do not determine the order total.

Coupons can be fixed or percentage based. Validation checks expiry, active state, minimum spend, usage limits and whether the customer has already redeemed the code. Percentage discounts are rounded down to whole rupees and can be capped. Delivery is INR 99 below an INR 1999 subtotal and free at or above that subtotal; the threshold uses the subtotal before the coupon discount.

Code example of conditional stock reservation

```
const result = await ProductModel.updateOne({
  productId: item.productId, active: true,
  variants: { $elemMatch: {
    size: item.size, stock: { $gte: item.quantity }
  } }
}, {
  $inc: {
    'variants.$.stock': -item.quantity,
    quantity: -item.quantity, revision: 1
  }
}, { session });
ensure(result.modifiedCount === 1,
  'Stock changed. Please refresh your bag and try again.', 409);
```

The filter checks size and sufficient stock in the same database update. A competing checkout cannot simply overwrite a previously read quantity. The update runs inside a MongoDB transaction together with coupon redemption and order creation. A failed step rolls back the transaction.

Retries and fulfilment

placeOrder uses requestId and a unique userId plus requestId index to make retries idempotent: the same submitted request returns its existing order. Orders retain product names, prices, images and customer/address snapshots so later catalogue edits do not rewrite purchase history.

The allowed progression is Pending to Processing to Shipped to Delivered. Pending or Processing can instead become Cancelled. Cancellation returns stock and releases the coupon within a transaction. Delivery records COD payment as collected. The admin dashboard counts collected revenue only from delivered orders marked collected.

Code reference backend/commerce.ts; backend/domain.ts; backend/models.ts; src/store/Checkout.tsx

8 Database models and stored information

MongoDB Atlas is the main database, accessed through Mongoose. The default application database name is shopDB, overridable with MONGO_DB_NAME. Atlas or another replica-set deployment is required because the implementation relies on multi-document transactions. Neon Auth is an optional identity service; the store does not use a separate PostgreSQL catalogue or order database.

Collection	Important stored fields
users	Name, unique normalized email, optional age, hidden password hash, role, auth provider, Neon identity, tokenVersion, writeRevision and embedded session records.
products	Unique productId, name, integer INR price, category, collection, colour, images, material, care, five size variants, total quantity, active flag and revision.
orders	Unique orderId, user reference, requestId, item and address snapshots, subtotal, discount, delivery, total, COD payment state and fulfilment state.
coupons	Unique code, discount type/value, minimum spend, cap, expiry, usage limit, usedCount, usedBy and active flag.
reviews	Product identifier, user reference, rating, text, attached image URLs and verifiedPurchase flag.
faceoffs	Author reference, title, caption, one to three labelled looks and voting closing time.
votes	Face-off reference, voter reference and zero-based selected option.
media	Unique mediaId, owner, attachment reference, byte length, WebP content type and hidden binary image data.
rate_limits	Hashed scope/window/IP key, counter and expiration time.
migrations	Completed catalogue migration identifiers and creation time.

Why documents and embedded fields fit this application

Product variants are embedded because five sizes belong to one product and are read together. Order items and shipping details are embedded snapshots because an order must preserve what was purchased. Reviews, votes and media are separate collections because they have their own ownership, uniqueness, paging and deletion rules. Sessions are embedded in the user to support bounded, revocable authentication.

Code reference backend/models.ts; backend/database.ts; backend/catalog-expansion.ts

9 Relationships indexes and consistency

Logical relationships

```
User       1 ---- many Orders
User       1 ---- many Reviews ---- 1 Product by productId
User       1 ---- many Faceoffs
Faceoff    1 ---- many Votes ---- 1 User
User       1 ---- many Media
Media      ---- attachedTo review ID or faceoff ID
Product    ---- embedded size variants
Order      ---- embedded purchased item snapshots
```

Mongoose references and product identifiers connect records; these are not relational foreign-key constraints. Application transactions and cleanup logic maintain the relationships. Public community responses expose the author name and ownership state while removing the author userId from the public record.

Indexes with direct business meaning

Index or rule	What it protects
Unique emailId and productId	No duplicate registered email or public product identifier.
Unique userId plus requestId	A repeated checkout request cannot create a second order.
Unique productId plus userId on reviews	Each member has one editable review for a product.
Unique faceoffId plus userId on votes	Each member has one vote per post, which may be changed.
Partial unique Neon issuer plus identity	The same verified identity is not attached to multiple users.
TTL for unattached media and rate counters	Draft images expire after about one hour; obsolete limit buckets are removed.

Concurrency and initialization

Admin product edits send a revision. The update filter must match the current revision and then increments it; a stale form receives a conflict instead of overwriting newer stock. Community writes use userWrite, which increments the user writeRevision inside a transaction before saving associated data. This helps coordinate them with account deletion.

Catalogue migrations use stable IDs and insert-only updates for missing products. A migrations record prevents repeated installation. Restarting does not refill sold inventory or recreate an archived product. Database setup creates indexes explicitly because automatic index creation is disabled for normal connections. Legacy disk uploads are migrated into MongoDB and checked before the old files are removed.

Code reference backend/models.ts; backend/commerce.ts; backend/app.ts; backend/user-write.ts; backend/database.ts; backend/migrate-media.ts

10 Password authentication and JWT sessions

Password registration and login

Registration validates name, email, age and an 8 to 128 character password, normalizes the email and always creates a customer. Passwords are stored as salted scrypt hashes. Each hash gets a random 16-byte salt and a 64-byte derived key. Verification derives the key again and uses timingSafeEqual. Password hashes are excluded from ordinary user queries and never returned in publicUser.

Code example of the application session

```
const token = jwt.sign({
  version: user.tokenVersion || 0
}, secret(), {
  subject: String(user._id), jwtid: id,
  expiresIn: '8h', issuer: 'forme', audience: 'forme-store'
});
res.cookie('forme_session', token, {
  httpOnly: true, secure: secureCookie(),
  sameSite: 'strict', maxAge: 8 * 3600000, path: '/'
});
```

The signed JWT identifies the user through sub and the session through jti. authenticate restricts verification to HS256 with the expected issuer and audience, then loads the current database user. It also checks tokenVersion and the embedded session ID and expiry. These database checks make sessions revocable and ensure the current role controls access; this is not a fully stateless JWT design.

Session lifecycle

- Sessions last eight hours, with at most three retained session slots. A fourth login displaces the oldest slot.
- The account page lists active sessions and can revoke an individual one. Ending the current session clears its cookie.
- Global logout increments tokenVersion and clears all recorded sessions. Password changes revoke prior sessions and issue a new current session.
- JWT cookies are HttpOnly and SameSite Strict. Secure is enabled for production or COOKIE_SECURE=true. The JWT is not kept in localStorage.

CSRF and authorization

Authenticated mutations require X-CSRF-Token. Its value is an HMAC derived from the session token and is compared safely on the server. Origin checks add another boundary. Administrator APIs require the database role admin, while customer data changes require ownership. Hiding an admin button is only a presentation check; the backend enforces access.

Code reference shared/password.cjs; backend/security.ts; backend/app.ts; src/store/api.ts; src/store/Sessions.tsx

11 Google authentication and account privacy

Optional Google sign in through Neon Auth

1. Auth.tsx shows Continue with Google when VITE_NEON_AUTH_URL is configured. neon.ts loads the Neon client on demand and starts its Google provider flow.

2. Google returns through Neon, then the browser reaches /auth/callback. The client requests a fresh Neon token and posts it to /api/auth/neon.

3. The backend fetches Neon public keys from its JWKS endpoint and uses jose to validate an EdDSA JWT, issuer, audience, expiry and required claims. It also requires a verified email and rejects a banned identity.

4. syncNeonUser reuses the same linked MongoDB user or creates a customer. The API then issues the ordinary FORME HttpOnly session, so subsequent store requests use the same local authorization system.

If an email already belongs to a password account, the user must sign in locally first and explicitly connect Google using the same email. The process preserves the MongoDB user ID, password, orders and role. A matching email alone does not silently link accounts. Google-only accounts need no invented password or age, and Google identity claims do not grant administrator privileges.

Account deletion and retained orders

For a password account, deletion requires the current password. A Google-only account confirms with a verified token for its linked identity. The storefront blocks deletion of administrator accounts. An in-transit Shipped order must be resolved first.

eraseAccount runs a transaction: it cancels Pending and Processing orders, restores their stock, releases relevant coupon redemptions and removes the user, sessions, reviews, face-offs, votes and uploaded media. Retained order records lose email and shipping address; the customer name and identifiers are replaced. Financial and purchased-item snapshots remain for store records.

Additional request protection

Input validation combines express-validator and explicit typed business helpers. Requests with nested Mongo operator, dotted or prototype keys are rejected. Security headers restrict framing, scripts and browser capabilities. Rate limits use MongoDB counters shared across server instances. These controls support the demo implementation; they do not imply that the application has received an independent security certification.

Code reference src/store/neon.ts; src/store/GoogleCallback.tsx; backend/neon-auth.ts; backend/privacy.ts; backend/security.ts; backend/validation.ts

12 Main API reference

All paths below are relative to the application origin. Public reads need no account. Protected writes use the session cookie and X-CSRF-Token; password login and verified provider exchange establish that session. Exact field rules remain in the route and domain modules.

Method and path	Access	Purpose
GET /api/products and /api/products/:id	Public	Active catalogue and single product.
POST /api/users and /api/login	Public with limits	Password registration and sign in.
POST /api/auth/neon	Verified provider token	Exchange or explicitly link Google identity.
GET /api/session	Signed in	Public user and session-bound CSRF token.
POST /api/logout	Signed in	Invalidate all recorded user sessions.
GET PUT PATCH DELETE /api/users/me	Owner	View, edit or close own account.
GET /api/users/me/dashboard	Owner	Order, review, face-off and vote counts.
GET /api/users/me/sessions	Owner	List active sessions.
DELETE /api/users/me/sessions/:id	Owner	Revoke an individual session.
POST /api/checkout/quote	Signed in	Current stock, coupon and total calculation.
POST and GET /api/orders	Owner	Place or list own orders.
PATCH /api/orders/:id/cancel	Owner	Cancel an eligible order.
POST /api/size-recommendation	Public with limits	Validated measurement-based estimate.
GET /api/products/:productId/reviews	Public	Ten reviews per page and rating statistics.
PUT DELETE /api/products/:productId/reviews/me	Owner	Save or remove own review.
GET POST /api/faceoffs	Public read signed write	Twelve posts per page or publish a post.
PUT /api/faceoffs/:id/vote	Signed in	Create or change a vote.
DELETE /api/faceoffs/:id	Owner or admin	Remove post, votes and photos.
POST /api/media and GET /api/media/:id	Signed upload public attachment	Upload photos or serve a published image.
/api/admin/dashboard products orders customers coupons	Admin	Store statistics and management operations.
GET /api/admin/community	Admin	Latest fifty reviews and fifty posts.
DELETE /api/admin/reviews/:id	Admin	Remove a review and its media.
GET /api/health	Public	Readiness response with database state.

Code reference backend/app.ts; backend/community.ts

Part Two Features and distinctive capabilities

13 Application feature inventory

FORME combines a working shopping flow with tools that help people decide what to buy, what size to choose and how an outfit looks to other members. Its distinction is this integrated feature set within the project, rather than a claim that these ideas are exclusive inventions in the wider ecommerce market.

Area	Implemented capabilities
Shopping	Search, category and collection filters, featured/price sorting, product details, local wishlist and bag, variant quantity checks and buy now.
Checkout	Indian shipping details, coupon quote, authoritative totals, COD order, retry protection and order confirmation.
Fit guidance	Required body measurements, optional chest/bust and hip, regular or roomier preference, explained size result and separate stock check.
Outfit suggestions	Up to three complementary in-stock pieces based on bag categories, collection and colour rules.
Photo reviews	Rating, review text, up to three images, own edit/delete, average score, pagination and delivered-purchase badge.
Outfit face-offs	One to three labelled looks, optional caption, explicit public consent, seven-day voting, changeable member vote and own posts.
Customer account	Profile, order history and cancellation, activity counts, password changes, Google linking, active sessions and account deletion.
Administrator studio	Collected revenue, order value, customer counts, seven-day trend, product editing/archival, size inventory, fulfilment and coupon controls.
Moderation and media	Admin content removal, browser photo reduction, server WebP re-encoding, attachment ownership and automatic draft expiration.

The connected shopper journey

A shopper discovers a product, reads customer experiences, checks a measurement-based estimate and chooses an available size. The bag suggests another category that completes the outfit. After checkout and delivery, a review can receive the verified badge. Separately, outfit face-offs let members seek community opinions on looks and return to the shop from the community page.

The community is based on member-created content. The catalogue contains sample products, but fabricated reviews or outfit posts are not seeded as real customer activity. Current face-offs use photo uploads; they do not automatically tag or sell every garment visible in a photo.

Code reference src/store/Catalogue.tsx; src/store/Wardrobe.tsx; src/store/Community.tsx; src/store/Account.tsx; src/store/Admin.tsx

14 Outfit face offs

What members can do

The /community page provides a public feed and a My face-offs view. A signed-in member publishes a title, optional caption and one to three uploaded looks, each with its own label. Even one photo is valid. The form requires agreement that the photos will be public. Labels remain associated with the correct image when another image is removed.

Each post closes seven days after creation. Other signed-in members choose an option and see vote counts and percentages. They may change their selection until closing. The author cannot vote on their own looks. Owners and administrators can delete a post; related votes and photos are removed.

Code example of one changeable vote

```
await VoteModel.updateOne({
  faceoffId: post._id,
  userId: who(req)._id
}, {
  $set: { option: req.body.option }
}, {
  upsert: true, session, runValidators: true
});
```

The filter identifies the member and post, so a later vote updates the existing selection instead of adding a second vote. A unique compound index reinforces that rule. The route also checks the closing time, prevents self-votes and verifies that the chosen option exists. It writes the post within the transaction to coordinate voting with closing or deletion.

Abuse controls and feed behavior

- Posts are paginated twelve at a time, with totals and each signed-in member's current choice.
- A member can create up to five currently counted new posts in a rolling day. A separate post-create limiter permits thirty requests per hour per IP scope.
- Voting has a sixty-per-minute IP limiter in addition to the overall API limit.
- Public responses include author display name, look labels, counts and ownership flags; the public author userId field is removed.
- Admin moderation displays the latest fifty posts. The current implementation does not include threaded comments, follower relationships or automatic image moderation.

Code reference src/store/Community.tsx; backend/community.ts; backend/validation.ts; backend/models.ts

15 Product reviews and photo handling

Reviews that connect to actual orders

Product detail pages show a one to five star score, text, customer photos, average rating and review count. Reviews load ten per page. Signed-in members can save, edit or delete their own single review for a product. Review text is validated at ten to two thousand characters and images are limited to three. The interface requests public-publishing consent.

```
const verifiedPurchase = !!await OrderModel.exists({
  userId,
  status: 'Delivered',
  'items.productId': productId
}).session(session);
```

The badge is derived from a delivered order containing the product when the review is saved. The browser cannot grant itself verified status. A member without a delivered purchase may still review, but does not receive the badge. A review written before delivery needs a later save to recompute this stored flag; delivery itself does not update all existing reviews.

Two stage image processing

1. Photos.tsx accepts JPEG, PNG or WebP originals under 6 MB, checks dimensions and reduces them with a browser bitmap and canvas before upload.

2. The upload endpoint accepts one to three encoded images under its 4 MB JSON request limit. Sharp decodes actual image bytes, restricts pixel count and rejects unsupported or animated input.

3. The backend rotates, resizes within 1200 by 1200 pixels, flattens transparency and re-encodes WebP. A second compression pass is available; saved bytes must stay at or below 250 KB per image. EXIF and GPS metadata are removed.

4. MongoDB stores each image with mediaId, ownerId and an initially empty attachedTo reference. Saving a review or post claims owned photos transactionally.

5. GET /api/media/:id serves only published attachments. Draft URLs remain private through a 404 response. Unattached uploads expire after about an hour through a partial TTL index.

The photo claim checks both ownership and the destination attachment. Someone cannot attach another member's image or reuse a claimed image in a different post. A batch is fully decoded before storing it, reducing partial-upload failures. Removing content also removes its media. Catalogue images are separate static files under public/catalog.

Code reference src/store/Wardrobe.tsx; src/store/Photos.tsx; backend/community.ts; backend/media.ts; backend/models.ts

16 Size recommendation logic

The size guesser is a transparent measurement guide, implemented in shared/sizing.ts and exposed through POST /api/size-recommendation. It uses an authored general XS to XL chart rather than camera analysis, virtual try-on or a trained fit model. Body measurements are processed for the response and are not persisted by the application.

Inputs and chart

Required fields are height in centimetres, weight in kilograms, waist circumference and shoulder width. Chest/bust and hip are optional. The shopper selects regular or relaxed fit. Shared field definitions keep the form limits and backend validation messages consistent.

Size	Chest cm	Waist cm	Shoulder cm
XS	81 to 86	66 to 71	38 to 40
S	89 to 94	74 to 79	40 to 42
M	97 to 102	81 to 86	42 to 44
L	104 to 109	89 to 94	44 to 46
XL	112 to 117	97 to 102	46 to 48

How the result is selected

Bottoms use waist as the scored dimension. Tops and layers use shoulder and waist, plus chest when supplied. For each dimension, the algorithm finds the first band whose upper value accommodates it, then chooses the largest required band. Measurements between listed bands go to the roomier band. Relaxed preference steps up one size where possible; XL produces a limitation note. A measurement above the chart returns no size.

```
const slot = SIZE_CHART.findIndex(row =>
  dimension.value <=
    row[dimension.key as 'waist' | 'shoulder' | 'chest'][1]
);
if (slot < 0) { beyond = true; continue; }
index = Math.max(index, slot);
```

Example and limits

For a top with shoulder 43 cm, waist 83 cm and chest 100 cm, all scored dimensions fit M. Relaxed preference moves the estimate to L. The result separately checks whether that size has stock. An unavailable M is not silently replaced with another stocked size.

Height supplies garment-length warnings, and weight provides context rather than an inferred circumference. Hip is optional context because this chart has no calibrated hip bands. Sets warn that the two pieces may need different sizes. Product-specific, brand-specific and women-specific charts would improve future accuracy; the current result remains a general estimate.

Code reference shared/sizing.ts; src/store/Wardrobe.tsx; backend/community.ts; docs/SIZING.md

17 Automatic cart suggestions

The bag drawer includes CartSuggestions, which calls shared/outfits.ts with the loaded products and product IDs already in the bag. This is a deterministic outfit-completion algorithm running in the browser. It recommends extra pieces; it does not add them automatically or predict a size for them.

Candidate filtering and ranking

Rule	Effect
Active with positive total stock	Exclude archived and sold-out products.
Not already in the bag	Avoid suggesting another copy of a selected product.
Compatible collection	If the bag has exactly one non-Unisex collection, include that collection and Unisex candidates.
Missing Bottoms with Tops or Layers	Add 10 points and explain that a bottom completes the outfit.
Missing Tops with Bottoms	Add 10 points and explain the complementary top.
Missing Layers	Add 8 points when the earlier complement rules do not apply.
Neutral colour	Add 2 points for a colour in the explicit neutral-colour list.
Colour differs from bag colours	Add 1 point for variation.
Ranking and limit	Higher score first, then lower price; return at most three pieces.

Code example of the final ordering

```
.map(product => ({ product, ...rank(product) }))
.filter(p => p.score >= 0)
.sort((a, b) =>
  b.score - a.score ||
  a.product.productPrice - b.product.productPrice
)
.slice(0, 3);
```

For example, a bag containing a top but no bottoms can receive an in-stock bottom with an explanatory reason. A layer can also appear if the bag lacks one. The shopper selects an available size and explicitly adds the recommendation. The ordinary addToBag checks still apply, and checkout validates current stock again.

What makes this useful and where it stops

Reasons are visible and easy to explain in a presentation. The rules need no training dataset, tracking history or external AI service. They use category, collection, colour and price; they do not understand occasion, actual garment images, personal taste or long-term purchase behavior. Empty bags have no suggestions. A future improvement could add occasion tags and feedback while preserving an explanation for each recommendation.

Code reference shared/outfits.ts; src/store/Wardrobe.tsx; src/store/Layout.tsx; src/store/StoreContext.tsx

18 Technical strengths and future scope

Strengths the team can explain with evidence

- Modern typed interface: React 19 with TypeScript, React Router, Tailwind 4 and Vite 8. These are the major versions declared by this project; they are not a claim that every dependency is the newest release.
- Clear state boundaries: shared React Context for session and shopping state, local hooks for forms, URL parameters for navigation and localStorage for device preferences.
- Revocable JWT authentication: HttpOnly cookies, live session and role checks, session limits, tokenVersion invalidation and CSRF headers.
- Transactional commerce: authoritative prices, conditional variant-stock updates, coupon reservation, order snapshots and idempotent retries.
- Concurrency protection: product revision checks prevent stale inventory edits; unique indexes protect reviews, votes, identities and checkout requests.
- Practical community media: browser size reduction, Sharp re-encoding, metadata removal, owned attachment references and MongoDB persistence across redeployments.
- Deployment-aware design: one frontend/API origin, cached database connections, MongoDB-backed rate limiting and TTL cleanup.
- Explainable recommendations: shared measurement rules and readable outfit-ranking reasons make the behavior testable and easy to present.
- Automated verification: repository audit, TypeScript checks, Node tests and frontend build are included in GitHub Actions.

Current boundaries and possible extensions

Checkout currently supports COD only. PayU or another gateway would require server-side payment initiation, verified callbacks or webhooks, idempotency and reconciliation. The current interface does not provide password reset or application email verification for local accounts, shipping-carrier tracking, cross-device bag synchronization or trained recommendation models.

Useful next features include password recovery, email verification, product-specific fit charts, a server-persisted wishlist and bag, occasion-based suggestions, member reporting of inappropriate posts, and fuller admin search/pagination. These are proposed extensions, separate from the implemented feature inventory.

The strongest presentation position is that FORME goes beyond basic product CRUD by combining safe inventory handling with social outfit decisions and explained fit guidance. It remains a college demonstration with illustrative product data; a public operation would also need ongoing monitoring, backups and operational processes.

Code reference package.json; backend/commerce.ts; backend/security.ts; backend/media.ts; shared/sizing.ts; shared/outfits.ts; .github/workflows/ci.yml

19 Setup configuration and validation

How a teammate runs the project

1. Use Node.js 22.x and install the locked dependencies from the repository root with npm ci. The frontend and backend do not need separate installations.
2. Copy .env.example to a private .env and configure MONGO_URI, a JWT_SECRET of at least 32 characters and the intended MONGO_DB_NAME. Use PORT=3000 locally and suitable COOKIE_SECURE settings for HTTP or HTTPS.
3. Configure MongoDB Atlas credentials and Network Access. Run npm run env:check and npm run db:setup against the intended project database.
4. For optional Google login, configure Neon Managed Better Auth and its Google provider, trusted redirects and matching NEON_AUTH_URL and VITE_NEON_AUTH_URL. The VITE value is public; secrets must never use a VITE_ prefix.
5. Run npm run dev and use the printed local URL. Provision a chosen administrator with ADMIN_EMAIL and npm run admin:setup. A newly generated password is saved privately, not embedded in React.
6. For standalone production use npm run build followed by npm start. For Vercel, use the repository root, npm ci, npm run build and dist; the API is invoked by the platform.

Verification performed for this guide

On 5 November 2026, npm run check completed successfully: the tracked-file audit checked 133 files, TypeScript reported no errors, all 22 tests passed and Vite generated the production frontend. These checks cover helper behavior and build correctness; they do not establish that every external service or deployed browser flow is working.

Command	Purpose
npm run check	Repository audit, TypeScript, unit tests and production build.
npm run test:atlas	Live commerce and account API checks through the existing server.
npm run test:media	MongoDB image bytes, ownership, publication and cleanup checks.
npm run test:neon	Identity synchronization and explicit linking checks.
npm run db:check	Credential-safe MongoDB diagnostics.

The database-writing live suites should use an isolated test environment. VERIFICATION.md records earlier live checks, but those are separate from the check run above. A real Google consent round trip and deployment acceptance still need environment-specific verification. Before publishing, follow the credential-rotation guidance in docs/DEPLOYMENT.md because removing formerly tracked secrets does not erase Git history.

Code reference package.json; .env.example; docs/DEPLOYMENT.md; VERIFICATION.md; scripts/*

20 Team demonstration and code reading guide

Suggested presentation order

1. Introduce FORME as a MERN clothing store that combines commerce, fit guidance and community opinion. Show the architecture diagram before the feature screens.
2. Browse the catalogue and filter a collection. Open a product, show its material and care information, choose a size and explain per-size stock.
3. Run the measurement guide and show that the recommended size and availability are separate. Explain the chart rule using the worked M-size example.
4. Add a product to the bag, inspect the complementary suggestions and choose a size for one. Explain the category scores and visible reasons.
5. Sign in, request a checkout quote and place a disposable COD order in the test environment. Explain server prices, transaction boundaries and retry protection.
6. Use admin to progress the order through Processing, Shipped and Delivered. Show when collected revenue changes and when the review can gain a verified badge.
7. Publish a review with a photo and an outfit face-off. Use a separate test member to vote, change the choice and demonstrate that self-voting is blocked.
8. Finish with session controls, moderation and the implemented-versus-future scope. Avoid deleting a real account or changing production inventory during a presentation.

Read the code in this order

Start with src/App.tsx and src/store/StoreContext.tsx to understand page composition and shared state. Follow src/store/api.ts into backend/app.ts to see the request boundary. Read backend/models.ts before backend/commerce.ts so the transaction fields are familiar. Then read backend/security.ts and backend/neon-auth.ts for authentication, followed by backend/community.ts and backend/media.ts for social content. Finish with shared/sizing.ts and shared/outfits.ts for the two recommendation algorithms.

Useful answers for a team discussion

MERN combines React for the interface, Express and Node for the API, and MongoDB for application documents. TypeScript is used across most frontend, backend and shared code. JWT verifies the signed request identity, while database session records allow immediate revocation and current role checks. Transactions ensure that stock, coupon use and order creation succeed together or are rolled back.

The recommendations use explicit rules with explainable results. Neon handles optional Google identity, while MongoDB stores the shop. COD is the implemented payment flow. Bag and wishlist entries persist on the current browser device, so signing in on a different device does not automatically restore them.

Reference documents in the repository

docs/FEATURE_STATUS.md summarizes feature status; docs/SIZING.md explains chart assumptions; docs/DEPLOYMENT.md covers hosting and Google setup; docs/CATALOG_IMAGE_SOURCES.json records catalogue image origins. README.md is the general introduction, but current code and the focused guides should resolve older statements about Google login, rate limits and startup behavior.